LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Name: Aditya Umashankar **SRN:** PESU1UG24AM335 **Section:** F

Project: Customizable Subscription Box Scheduler

1. Scenario Review

The Customizable Subscription Box Scheduler is a subscription-based portal where subscribers customize monthly product boxes based on preference tags. Subscribers can select and swap products and can pause or skip upcoming billing cycles before the applicable deadline. The system also supports fulfillment operations by allowing the Fulfillment Lead to generate fulfillment manifests and shipping labels for finalized subscription boxes.

Key Functional Requirements: manage preference tags; select monthly box items; swap items before the 48-hour deadline; pause or skip a billing cycle; generate fulfillment manifests; generate shipping labels; and process monthly subscription billing.

Key Non-Functional Requirements: the fulfillment manifest generator must export shipping labels for 10,000 monthly boxes in under one minute; subscriber and payment information must be protected; and the system should provide reliable access.

Architectural Challenges: performance during peak fulfillment processing, security of subscriber/payment data, scalability as subscriptions increase, and maintainability through separation of responsibilities.

2. Architectural Style Analysis

Architecture	Advantages	Disadvantages
Layered	Clear separation of presentation, business logic, payment, fulfillment, and data responsibilities; easy to understand and maintain	Changes may pass through multiple layers and can increase perfor
Microservices	Customization, billing and fulfillment can be independently developed, scaled, and deployed	Higher development and operational complexity, network communication a
Client-Server	Simple centralized design with centralized business logic and data	Greater dependency on the server and potential scalability bottlen

3. Architecture Selection

Selected Architecture: Layered Architecture

Layered Architecture is selected because the system has clearly separated responsibilities for subscriber interaction, subscription management, payment processing, data storage, and fulfillment. This separation makes the system easier to understand, develop, test, and maintain.

A layered design is also appropriate because the current system is a centralized subscription portal rather than a collection of highly independent services. It avoids the operational complexity of microservices while still allowing major functional areas to be represented as separate components.

Security advantage: sensitive subscriber and payment data can be isolated within the appropriate service and data layer, while the presentation layer does not directly access the database. Access can therefore be controlled through the business/service layer.

Performance benefit: the fulfillment component can process finalized subscription data and generate manifests and shipping labels separately from the user-facing layer, helping heavy fulfillment operations avoid directly affecting subscriber interactions.

4. Component Identification

Component	Responsibility
Subscriber Portal	Subscriber interface for preferences, box selection, item swapping, and pause/skip requests.
Order Manager	Manages subscription-box customization, validates deadlines, and coordinates payment and fulfillment.
Payment Service	Processes monthly billing and payment authorization/status.

Subscription Database	Stores subscriber, preference, product, box-selection, subscription and fulfillment data.
Fulfillment Manager	Generates fulfillment manifests and shipping labels for finalized boxes.

5. Interfaces and Data Flow

The Subscriber Portal communicates with the Order Manager through a REST API for subscription requests. The Order Manager communicates with the Payment Service through a payment-processing API. It accesses the Subscription Database using SQL/data-access operations for subscription and product information. The Order Manager sends finalized box data to the Fulfillment Manager through a fulfillment API, and the Fulfillment Manager accesses the database for fulfillment data.

Conclusion

The Customizable Subscription Box Scheduler uses a Layered Architecture to separate subscriber interaction, business logic, payment processing, fulfillment operations, and data storage. This structure provides clear separation of responsibilities while supporting security, maintainability, and efficient fulfillment processing. The component model demonstrates the major system components and the interfaces through which they communicate.